

DSAI 512 Fall 2025

HW #5

Yiğit Ateş - 2025776009

Institute for Data Science and Artificial Intelligence, Boğaziçi University

I. PROBLEM 1

Part a)

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix}, \quad y = \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix}$$

$$x_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad x_1 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \quad X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix}$$

$$x'_2 = \begin{bmatrix} 4 \\ 8 \\ 12 \end{bmatrix}, \quad x'_3 = \begin{bmatrix} 6 \\ 12 \\ 18 \end{bmatrix}$$

$$\tilde{X} = \begin{bmatrix} 1 & 2 & 4 & 6 \\ 1 & 4 & 8 & 12 \\ 1 & 6 & 12 & 18 \end{bmatrix}$$

i) Show that $X^\top X$ is invertible:

$$X^\top = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix}$$

$$X^\top X = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix}$$

$$X^\top X = \begin{bmatrix} 3 & 12 \\ 12 & 56 \end{bmatrix}$$

To check for invertibility, we calculate the determinant:

$$\det(X^\top X) = (3)(56) - (12)(12) = 24$$

Since $\det(X^\top X) \neq 0$, the matrix $X^\top X$ is **invertible**.

ii) Show that $\tilde{X}^\top \tilde{X}$ is not invertible: We define the matrix $\tilde{X} = [x_0 \ x_1 \ x'_2 \ x'_3]$. From the problem definition, we know:

$$x'_2 = 2x_1 \quad \text{and} \quad x'_3 = 3x_1$$

This means the columns of $\tilde{X}$ are linearly dependent. Consequently, the columns of the product $\tilde{X}^\top \tilde{X}$ are also linearly dependent. For any square matrix, if the columns are linearly dependent, the matrix is singular and its determinant is zero.

$$\det(\tilde{X}^\top \tilde{X}) = 0$$

Since the determinant is zero, the matrix is **not invertible**.

iii) Show that the predictions $\tilde{X}w$ are unique: The problem statement references parameters w_2, w_3, w_4 . Therefore, I assume the weights start from w_1 (corresponding to x_0) rather than w_0 .

Let the weight vector for the matrix $\tilde{X}$ be $w = [w_1, w_2, w_3, w_4]^\top$. The prediction vector $\hat{y}$ is given:

$$\hat{y} = \tilde{X}w = w_1x_0 + w_2x_1 + w_3x'_2 + w_4x'_3$$

Substituting the known linear dependencies $x'_2 = 2x_1$ and $x'_3 = 3x_1$:

$$\hat{y} = w_1x_0 + w_2x_1 + w_3(2x_1) + w_4(3x_1)$$

We can group the terms associated with x_1 :

$$\hat{y} = w_1x_0 + (w_2 + 2w_3 + 3w_4)x_1$$

Defining the new weight as $w_{\text{sum}} = w_2 + 2w_3 + 3w_4$. The prediction equation becomes:

$$\hat{y} = w_1x_0 + w_{\text{sum}}x_1$$

This is exactly the same form as the linear model for the original matrix $X = [x_0 \ x_1]$. The problem simplifies to finding the unique constants w_1 and w_{sum} that minimize the squared error for the independent columns x_0 and x_1 .

Since we proved in part (i) that $X^\top X$ is invertible, the solution for the coefficients of x_0 and x_1 is unique. Thus, the optimal values for w_1 and w_{sum} are unique.

Consequently, even though the individual values of w_2, w_3, w_4 can vary, the resulting prediction vector $\hat{y}$ is unique. This demonstrates that for linearly dependent cases we can successfully predict the correct (unique) value; however, we cannot rely on the weights as they will yield the unique solution that satisfies the w_{sum} value.

Part b)

i) Show that the columns of $\tilde{X}$ are linearly dependent and hence $\tilde{X}^\top \tilde{X}$ is not invertible: Given the relationship $x_3 = x_1 + x_2$, we directly see that the third column x_3 is a linear combination of the first two columns. By definition, the set of columns are linearly dependent.

Because the columns are linearly dependent, the matrix $\tilde{X}$ does not have full column rank. Specifically, although $\tilde{X}$ has 3 columns, its rank is only 2.

For the square matrix $\tilde{X}^\top \tilde{X}$ to be invertible, $\tilde{X}$ must have full column rank. Since $\text{rank}(\tilde{X}) < 3$, the matrix $\tilde{X}^\top \tilde{X}$ is singular. Therefore, its determinant is 0 and it is **not invertible**.

ii) *Argue that the least-squares problem has infinitely many minimizers:* From part (i), we established that the columns are linearly dependent with the relation $x_1 + x_2 - x_3 = 0$. This implies that the vector $v = [1, 1, -1]^\top$ lies in the null space of $\tilde{X}$:

$$\tilde{X}v = 1 \cdot x_1 + 1 \cdot x_2 - 1 \cdot x_3 = \mathbf{0}$$

We can add any multiple of v to our weights without changing the prediction.

Let $\tilde{w}^*$ be any specific solution that minimizes the error. We can construct a new weight vector $\tilde{w}'$ by adding an arbitrary amount α of the null vector v :

$$\tilde{w}' = \tilde{w}^* + \alpha v \quad \text{for any } \alpha \in \mathbb{R}$$

We verify that this new weight vector produces the exact same prediction:

$$\tilde{X}\tilde{w}' = \tilde{X}(\tilde{w}^* + \alpha v) = \tilde{X}\tilde{w}^* + \alpha(\underbrace{\tilde{X}v}_{\mathbf{0}}) = \tilde{X}\tilde{w}^*$$

Since the prediction $\tilde{X}\tilde{w}'$ is identical to $\tilde{X}\tilde{w}^*$, the squared error remains the same minimal value. Because α can be any real number, there are infinitely many such vectors $\tilde{w}'$. Thus, the problem has infinitely many minimizers.

iii) *Prove that all minimizers produce the same predictions:* From part (ii), we know that the problem has infinite minimizers. Let $\tilde{w}^*$ be one specific minimizer. Any other minimizer $\tilde{w}$ can be written as:

$$\tilde{w} = \tilde{w}^* + v$$

where v is a vector in the null space of $\tilde{X}$.

The prediction produced by any such minimizer is:

$$\hat{y} = \tilde{X}\tilde{w} = \tilde{X}(\tilde{w}^* + v)$$

Distributing the matrix multiplication:

$$\hat{y} = \tilde{X}\tilde{w}^* + \tilde{X}v$$

Since v is in the null space, $\tilde{X}v = 0$. Therefore:

$$\hat{y} = \tilde{X}\tilde{w}^*$$

This shows that adding the null vector v has no effect on the output. Every possible minimizer $\tilde{w}$ produces the exact same prediction vector as $\tilde{w}^*$. Thus, the prediction is unique. In addition to that, by definition, the vector in a subspace that minimizes the distance to y is the orthogonal projection of y onto that subspace. Thus Xw is the orthogonal projection of y onto $\text{col}(X)$. Finally, because X is given to have full column rank, the linear system $Xw = \hat{y}$ has exactly one solution, proving that w is unique.

II. PROBLEM 2

Part a)

Substituting $p(y_i|x_i, w) = \sigma(y_i w^\top x_i)$:

$$L(w) = \prod_{i=1}^N \sigma(y_i w^\top x_i)$$

To simplify, we take the natural logarithm of the likelihood:

$$\ln L(w) = \ln \left(\prod_{i=1}^N \sigma(y_i w^\top x_i) \right) = \sum_{i=1}^N \ln \sigma(y_i w^\top x_i)$$

We use $\ln \sigma(z) = -\ln(1 + e^{-z})$. Here, let $z = y_i w^\top x_i$.

$$\ln L(w) = \sum_{i=1}^N -\ln(1 + e^{-y_i w^\top x_i})$$

The goal of learning is to maximize the likelihood, which is equivalent to minimizing the negative log-likelihood. The average in-sample error is defined as the negative log-likelihood normalized by the number of samples N :

$$E_{in}(w) = -\frac{1}{N} \ln L(w)$$

Substituting our expression for $\ln L(w)$:

$$E_{in}(w) = -\frac{1}{N} \sum_{i=1}^N -\ln(1 + e^{-y_i w^\top x_i})$$

$$E_{in}(w) = \frac{1}{N} \sum_{i=1}^N \ln(1 + e^{-y_i w^\top x_i})$$

Part b)

$$E_{in}(w) = \frac{1}{N} \sum_{i=1}^N \ln(1 + e^{-y_i w^\top x_i})$$

Let $z_i = -y_i w^\top x_i$. The loss for a single sample is $J_i(w) = \ln(1 + e^{z_i})$. Using the chain rule, we compute the gradient:

$$\nabla_w J_i(w) = \frac{\partial J_i}{\partial z_i} \nabla_w z_i$$

First, the derivative with respect to the exponent z_i :

$$\frac{\partial}{\partial z_i} \ln(1 + e^{z_i}) = \frac{1}{1 + e^{z_i}} \cdot e^{z_i} = \frac{e^{z_i}}{1 + e^{z_i}} = \sigma(z_i)$$

Next, the gradient of z_i with respect to w :

$$\nabla_w (-y_i w^\top x_i) = -y_i x_i$$

Combining these terms:

$$\nabla_w J_i(w) = \sigma(z_i)(-y_i x_i) = \sigma(-y_i w^\top x_i)(-y_i x_i)$$

Averaging over all samples, the full gradient is:

$$\nabla_w E_{in}(w) = -\frac{1}{N} \sum_{i=1}^N y_i x_i \sigma(-y_i w^\top x_i)$$

Part c)

```

1 import numpy as np
2
3 def sigmoid(z):
4     return 1 / (1 + np.exp(-z))
5
6 def logistic_loss(w, X, y):
7     z = y * (X @ w)
8     return np.mean(np.log(1 + np.exp(-z)))
9
10 def logistic_grad(w, X, y):
11     N = X.shape[0]
12     z = y * (X @ w)
13     probs = sigmoid(-z)
14
15     grad = -(X.T @ (y * probs)) / N
16
17     return grad

```

III. ACKNOWLEDGEMENTS

Gemini 2.5 Pro was used to assist in converting mathematical notations and Python codes to LaTeX format as well as getting some hints for the solutions of the Problem 1.b.ii, 1.b.iii, and 2.b.